

Fetching Data & Eloquent

As I mentioned in the last video, Eloquent is an ORM or Object Relational Mapper, which is a powerful tool that allows us to interact with the database. In this lesson, we will learn how to use Eloquent to fetch data from the database.

Generating a Model

Let's re-create the Job model. We can actually generate a new model using the `make:model` command. This command will create a new model in the `app/Models` folder.

```
php artisan make:model Job
```

This will create a file at `app/Models/Job.php` that looks like this:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Job extends Model
{
    use HasFactory;
}
```

This is a basic model that extends the `Model` class. The `Model` class is provided by Eloquent and it has a lot of useful methods that we can use to interact with the database. The `HasFactory` trait is used to generate factories for the model. We will learn more about factories in a later lesson.

Create Model & Migration

I also want to mention that you can create a model and a migration at the same time using the `make:model` command. This is useful if you want to create a model and a migration for a new table. You can do this by passing the `--migration` flag to the `make:model` command.

You don't have to do this because we already created the migration, but I wanted to show you how to do it.

```
php artisan make:model Job --migration
```

This would create a model at `app/Models/Job.php` and a migration at `database/migrations/2022_01_01_000000_create_jobs_table.php`.

Specifying the Table Name

By default, it will look for a table with the same name as the model, but with the `s` suffix. In this case, it will look for a table called `jobs`. We don't want that since our table is called `job_listings`. We can specify the table name by adding a protected property to the model:

```
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
class Job extends Model
{
    protected $table = 'job_listings';
}
```

Now we can use this model to fetch data from the database.

Mass Assignment

Eloquent provides a way to protect against mass assignment vulnerabilities. This is a security feature that prevents users from modifying columns that they shouldn't be able to modify. To use this feature, we need to specify the columns that we want to allow mass assignment for. Mass assignment is when we pass an array of data to a model and the model will automatically assign the data to the columns in the database. We want to be able to mass assign most fields, for now, we just need to add the title and description fields.

```
class Job extends Model
{
    use HasFactory;

    protected $table = 'job_listings';
}
```

Fetching Data

The code that we have in the controller should still work because we are still using the `Job` model and the `all` method. The difference is that now we are fetching data from the database instead of a hardcoded array.

Arrow Syntax

In your `resources/views/jobs/index.blade.php` file, you can update the code to use the arrow syntax to access the properties of the job object. \

```
<x-layout>
  <ul>
    @forelse($jobs as $job)
      <li>{{ $job->title }}</li>
    @empty
      <li>No jobs found</li>
    @endforelse
  </ul>
</x-layout>
```

I am also going to just use `with()` now that we are only passing in a single variable. This is a little cleaner.

```
public function index(): View
{
    $jobs = Job::all();
    return view('jobs/index')->with('jobs', $jobs);
}
```

There is no data in the database yet. Let's add some. You could do this with PG Admin or the shell. I am going to use Tinker. We will do this in the next lesson.